UNIVERSIDAD TECNOLOGICA NACIONAL

Trabajo Práctico Integrador
Sistema de Gestión Hospitalaria

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.
Organización Empresarial

Materia
Bases de Datos I

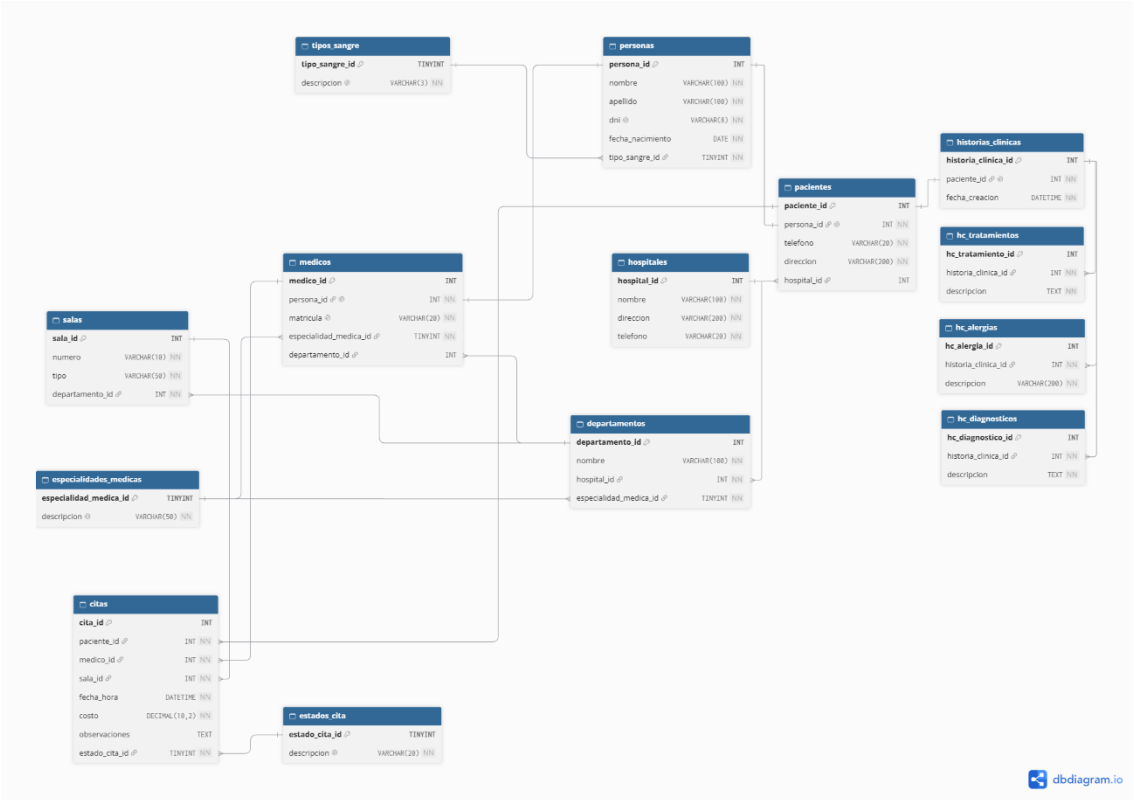
Docente
Carcaño, Nicolás

Alumno
Emperador, Javier

07 de noviembre de 2026

Etapa 1: Modelado y Constraints

Diagrama Entidad-Relación



El diagrama fue generado en dbdiagram.io, aquí el link para poder visualizarlo con más detalle: diagrama

Modelo relacional

- Tablas de catálogo
  - tipos_sangre (tipo_sangre_id, descripcion **NN UQ**)
  - especialidades_medicas (especialidad_medica_id, descripcion **NN UQ**)
  - estados_cita (estado_cita_id, descripcion **NN UQ**)

- Estructura del hospital
    - hospitales (hospital_id, nombre **NN**, direccion **NN**, telefono **NN**)
    - departamentos (departamento_id, nombre **NN**, hospital_id **NN FK**, especialidad_medica_id **NN FK**)
    - salas (sala_id, numero **NN**, tipo **NN**, departamento_id **NN FK**)

- Personas
    - personas (persona_id, nombre **NN**, apellido **NN**, dni **NN UQ**, fecha_nacimiento **NN**, tipo_sangre_id **NN FK**)
        - **dni**: VARCHAR(8), solo dígitos. CHECK: **dni REGEXP '^[0-9]{7,8}$'**
        - **fecha_nacimiento**: debe ser fecha pasada. Validación a nivel de aplicación (MySQL no permite **CURDATE()** en CHECK constraints).
    - pacientes (paciente_id, persona_id **NN UQ FK**, telefono **NN**, direccion **NN**, hospital_id **FK**)
    - medicos (medico_id, persona_id **NN UQ FK**, matricula **NN UQ**, especialidad_medica_id **NN FK**, departamento_id **FK**)

- Citas
    - citas (cita_id, paciente_id **NN FK**, medico_id **NN FK**, sala_id **NN FK**, fecha_hora **NN**, costo **NN**, observaciones, estado_cita_id **NN FK**)
        - **costo**: DECIMAL(10,2). CHECK: **costo >= 0**

- Historia clínica
    - historias_clinicas (historia_clinica_id, paciente_id **NN UQ FK**, fecha_creacion **NN**)
    - hc_diagnosticos (hc_diagnostico_id, historia_clinica_id **NN FK**, descripcion **NN**)
    - hc_tratamientos (hc_tratamiento_id, historia_clinica_id **NN FK**, descripcion **NN**)
    - hc_alergias (hc_alergia_id, historia_clinica_id **NN FK**, descripcion **NN**)

Tablas relacionales

Tabla origen	FK	Tabla destino	Cardinalidad
personas	tipo_sangre_id	tipos_sangre	N:1
departamentos	hospital_id	hospitales	N:1
departamentos	especialidad_medica_id	especialidades_medicas	N:1
salas	departamento_id	departamentos	N:1
pacientes	persona_id	personas	1:1
pacientes	hospital_id	hospitales	N:1
medicos	persona_id	personas	1:1
medicos	especialidad_medica_id	especialidades_medicas	N:1
medicos	departamento_id	departamentos	N:1
citas	paciente_id	pacientes	N:1
citas	medico_id	medicos	N:1
citas	sala_id	salas	N:1
citas	estado_cita_id	estados_cita	N:1
historias_clinicas	paciente_id	pacientes	1:1
hc_diagnosticos	historia_clinica_id	historias_clinicas	N:1
hc_tratamientos	historia_clinica_id	historias_clinicas	N:1
hc_alergias	historia_clinica_id	historias_clinicas	N:1

Decisiones de diseño

- Los enums del sistema Java (TipoSangre, EspecialidadMedica, EstadoCita) se modelaron como tablas de catálogo con FK, en lugar de columnas con CHECK. Esto permite agregar nuevos valores sin modificar el esquema y genera un modelo más extensible.
- **dni** se validó con REGEXP en un CHECK constraint en lugar de usar un tipo numérico, para preservar los ceros a la izquierda en DNIs de 7 dígitos.
- La fecha de nacimiento no usa **CURDATE()** en CHECK porque MySQL no lo permite. La validación se delega a la capa de aplicación.
- Paciente y Médico se modelaron como subtipos de Persona (relación 1:1 con **persona_id UQ FK**) para evitar duplicar atributos personales.

Validación de constraints

Scripts usados: **01_esquema.sql** y **02_catalogos.sql**

#	Tipo	Qué se prueba	Error esperado
1	Correcta	Carga completa: catálogos, hospital, personas, cita	Sin error
2	Correcta	Segundo paciente con datos válidos distintos	Sin error
3	Errónea	DNI duplicado (`30123456`)	`Duplicate entry` — viola `uq_personas_dni`
4	Errónea	Tipo de sangre inválido (`'XX'`)	`Check constraint` — viola `chk_tipos_sangre_desc`

Nota: Me parece que las capturas son demasiadas y aumentan en cantidad aún más a partir de la Etapa 3, por ende, se opta por adjuntar las capturas en una carpeta junto con los scripts para no hacer muy largo este informe.

Etapa 2: Generación y carga

Scripts usados: **03_carga_masiva.sql**

Descripción conceptual del mecanismo
Se generaron ~10.300 registros usando exclusivamente mecanismos nativos de SQL, sin datos externos ni scripts de aplicación.

- WITH RECURSIVE - generación de series numéricas
  Se usó una CTE recursiva para generar una secuencia de 1 a 2.000 números, materializada en una tabla temporal (MySQL Workbench no acepta **WITH RECURSIVE** seguido directamente de **INSERT INTO**). Cada número sirvió de base para construir atributos de personas mediante funciones de cadena y aritmética modular.

- CROSS JOIN - multiplicación de conjuntos
  Se aplicó para generar departamentos (hospitales × especialidades) y salas (departamentos × {1,2,3}): con 6 hospitales y 12 especialidades se obtienen 72 departamentos; con 3 salas por departamento se obtienen 216 salas en un único **INSERT**.

- INSERT ... SELECT - propagación encadenada
  Cada tabla dependiente se cargó seleccionando de la anterior, respetando el orden de dependencia de las FKs: **personas** → **medicos** / **pacientes** → **historias_clinicas** → **hc_diagnosticos** → **citas**.

- ROW_NUMBER() y aritmética modular — distribución de FKs sin gaps
  Para asignar FKs de forma distribuida sin asumir IDs contiguos, se numeró cada fila con **ROW_NUMBER()** y se usó **MOD** para elegir una fila real del conjunto destino. Esto evita el problema de **MOD + COUNT + 1** que falla cuando hay gaps en los IDs.

Distribución de los registros

Tabla	Mecanismo principal	Registros aprox.
hospitales	INSERT VALUES	6
departamentos	CROSS JOIN	72
salas	CROSS JOIN	216
personas	WITH RECURSIVE + tabla tmp	2.001
medicos	INSERT ... SELECT	500
pacientes	INSERT ... SELECT	1.500
historias_clinicas	INSERT ... SELECT	1.501
hc_diagnosticos	CROSS JOIN + SELECT	3.002
citas	INSERT ... SELECT	1.500
Total		**~10.298**

Verificaciones de consistencia

Se ejecutaron 5 verificaciones de FK huérfanas al final del script. Todas retornaron 0.

```sql
SELECT 'hospitales' AS tabla, COUNT(*) AS total FROM hospitales
UNION ALL
SELECT 'departamentos', COUNT(*) FROM departamentos
UNION ALL
SELECT 'salas', COUNT(*) FROM salas
UNION ALL
SELECT 'personas', COUNT(*) FROM personas
UNION ALL
SELECT 'medicos', COUNT(*) FROM medicos
UNION ALL
SELECT 'pacientes', COUNT(*) FROM pacientes
UNION ALL
SELECT 'historias_clinicas', COUNT(*) FROM historias_clinicas
UNION ALL
SELECT 'hc_diagnosticos', COUNT(*) FROM hc_diagnosticos
UNION ALL
SELECT 'citas', COUNT(*) FROM citas;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tabla	total
hospitales	5
departamentos	60
salas	180
personas	2001
medicos	500
pacientes	1500
historias_clinicas	1499
hc_diagnosticos	2998
citas	1499

```sql
-- FK huérfanas — deben retornar 0 en todos los casos
SELECT 'pacientes sin persona' AS verificacion, COUNT(*) AS resultado
FROM pacientes p LEFT JOIN personas pe ON p.persona_id = pe.persona_id
WHERE pe.persona_id IS NULL
UNION ALL
SELECT 'medicos sin persona', COUNT(*)
FROM medicos m LEFT JOIN personas pe ON m.persona_id = pe.persona_id
WHERE pe.persona_id IS NULL
UNION ALL
SELECT 'citas sin paciente', COUNT(*)
FROM citas c LEFT JOIN pacientes pa ON c.paciente_id = pa.paciente_id
WHERE pa.paciente_id IS NULL
UNION ALL
SELECT 'citas sin medico', COUNT(*)
FROM citas c LEFT JOIN medicos m ON c.medico_id = m.medico_id
WHERE m.medico_id IS NULL
UNION ALL
SELECT 'historias sin paciente', COUNT(*)
FROM historias_clinicas hc LEFT JOIN pacientes pa ON hc.paciente_id = pa.paciente_id
WHERE pa.paciente_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

verificacion	resultado
pacientes sin persona	0
medicos sin persona	0
citas sin paciente	0
citas sin medico	0
historias sin paciente	0

Etapa 3: Consultas avanzadas y reportes

Scripts usados: **04_indices.sql**, **05_consultas.sql** y **06_explain.sql**

Vista

vista_citas_detalle — Centraliza todos los datos de una cita: paciente, médico, especialidad, sala, departamento, hospital y estado. Evita repetir los mismos JOINs en cada consulta del sistema y simplifica el acceso desde la capa de aplicación.

Consultas

- **Q1** - Resumen de citas por hospital y departamento (JOIN múltiple)
  Tablas: **citas** → **salas** → **departamentos** → **hospitales** + **estados_cita**
  Permite al administrador ver la actividad de cada hospital: citas por departamento, estados y facturación. Útil para detectar hospitales con baja actividad o problemas de facturación.

- **Q2** - Plantel médico por hospital y departamento (JOIN con auxiliares)
  Tablas: **medicos** → **personas** + **especialidades_medicas** + **departamentos** → **hospitales**
  Muestra el plantel médico organizado por hospital y departamento. Permite verificar distribución de especialidades y detectar departamentos sin médicos asignados.

- **Q3** - Citas completadas por mes y especialidad (GROUP BY + HAVING)
  Tablas: **citas** → **medicos** → **especialidades_medicas** + **estados_cita**
  Reporte mensual por especialidad. Filtra con **HAVING COUNT > 5** para excluir especialidades con actividad marginal. Permite detectar tendencias y tomar decisiones sobre asignación de recursos.

- **Q4** - Pacientes con historia clínica, pero sin citas registradas (subconsulta NOT IN)
  Tablas: **pacientes** → **personas** + **historias_clinicas** / subconsulta sobre **citas**
  Identifica pacientes que tienen historia clínica pero nunca agendaron una cita. Útil para analizar retención y seguimiento de pacientes.

Medición de rendimiento con y sin índice

Metodología: 3 corridas por variante, se reporta la mediana. Se usó **EXPLAIN** para analizar el plan de ejecución. Índice creado: **idx_citas_fecha_hora** sobre **citas.fecha_hora**.

M1: Igualdad: **WHERE fecha_hora = '2025-06-15 10:00:00'**

Corrida	Con índice	Sin índice
1	< 1 ms	< 1 ms
2	< 1 ms	< 1 ms
3	< 1 ms	< 1 ms
Mediana	< 1 ms	< 1 ms

EXPLAIN con índice: **type = ref**, key = **idx_citas_fecha_hora**, rows = 1. EXPLAIN sin índice: **type = ref**, key = **fk_citas_estado**, rows = 374.

M2: Rango: **WHERE fecha_hora BETWEEN '2025-01-01' AND '2025-03-31'**

Corrida	Con índice	Sin índice
1	15 ms	< 1 ms
2	16 ms	< 1 ms
3	< 1 ms	< 1 ms
Mediana	15 ms	< 1 ms

M3: JOIN: **citas ↔ medicos ↔ personas**

Corrida	Con índice FK	Sin índice FK
1	< 1 ms	< 1 ms
2	< 1 ms	< 1 ms
3	< 1 ms	< 1 ms
Mediana	< 1 ms	< 1 ms

Conclusión: Con ~1.500 registros en **citas** todos los tiempos fueron menores a 1 ms. La excepción fue M2 con índice (mediana 15 ms), donde el overhead del índice B-tree superó al full scan por el pequeño volumen de datos. Esto es esperable: el impacto real de los índices sería visible a partir de 200.000–500.000 registros, rango recomendado por el enunciado para mediciones representativas.

Nota: Mismo caso que las capturas de la etapa 1.

Etapa 4: Seguridad e integridad

Scripts usados: **07_vistas.sql** y **08_seguridad.sql**

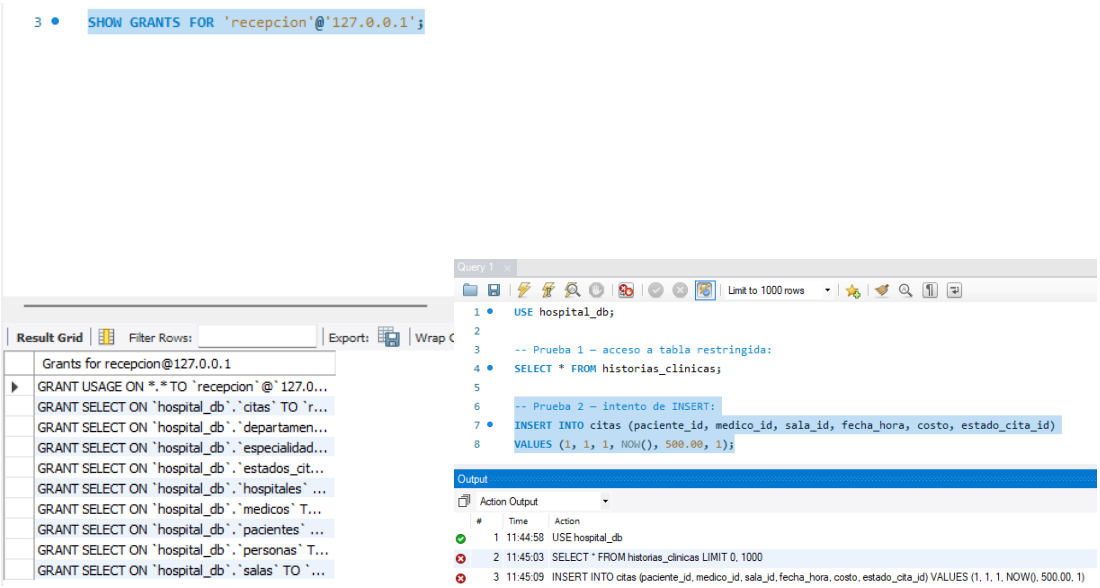
Usuario con privilegios mínimos

Se creó el usuario **recepcion@127.0.0.1** con **SELECT** únicamente sobre las tablas operativas del sistema (pacientes, personas, citas, médicos, especialidades, estados, hospitales, departamentos, salas). No tiene acceso a historias clínicas, diagnósticos, tratamientos, alergias ni tipos de sangre, y no puede modificar ningún dato.

Nota: El usuario se creó con host **127.0.0.1** (no **localhost**) porque MySQL Workbench se conecta por TCP y MySQL los trata como hosts distintos.

Prueba de acceso restringido:

- **SELECT * FROM historias_clinicas** → Error 1142: SELECT command denied
- **INSERT INTO citas …** → Error 1142: INSERT command denied



Prueba de integridad:

#	Qué se prueba	Error generado
1	INSERT con DNI duplicado	Error 1062: Duplicate entry — viola `uq_personas_dni`
2	INSERT con `estado_cita_id = 999`	Error 1452: Cannot add or update a child row — viola `fk_citas_estado`

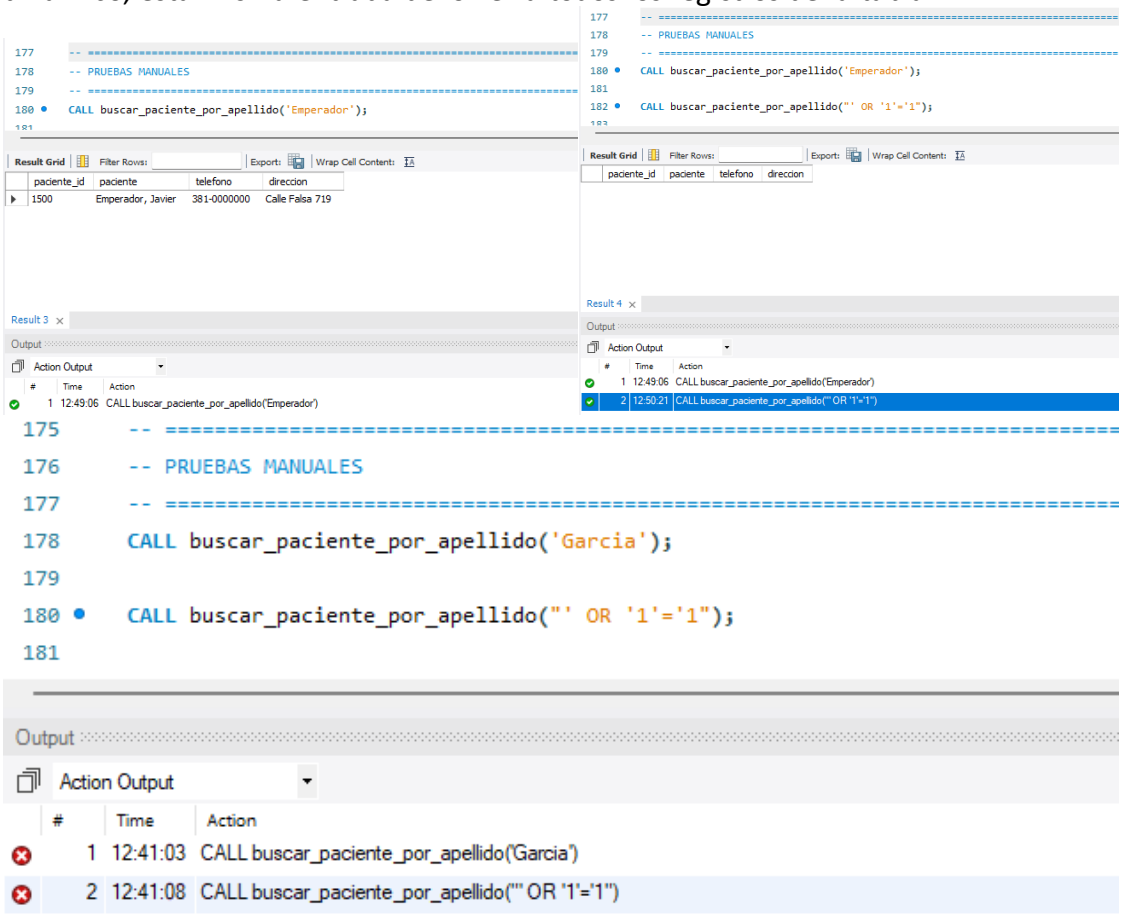
Consulta segura contra SQL injection

Se creó **buscar_paciente_por_apellido(p_apellido VARCHAR(100))**. Al usar un parámetro tipado en lugar de concatenación de strings, el valor ingresado siempre se trata como dato, nunca como código SQL.

Nota: Fue necesario agregar **COLLATE utf8mb4_0900_ai_ci** en el WHERE para resolver un conflicto de collation entre la base (**utf8mb4_unicode_ci**) y el servidor (**utf8mb4_0900_ai_ci**).

Prueba anti-inyección:

Resultado: 0 filas. La cadena maliciosa se trata como dato, no como SQL. Con SQL dinámico, esta misma entrada devolvería todos los registros de la tabla.




```
175     -- ============================================================================
176     -- PRUEBAS MANUALES
177     -- ============================================================================
178     CALL buscar_paciente_por_apellido('Garcia');
179
180 •   CALL buscar_paciente_por_apellido("' OR '1'='1");
181
```

Output

Action Output

#	Time	Action
1	12:41:03	CALL buscar_paciente_por_apellido('Garcia')
2	12:41:08	CALL buscar_paciente_por_apellido("' OR '1'='1")

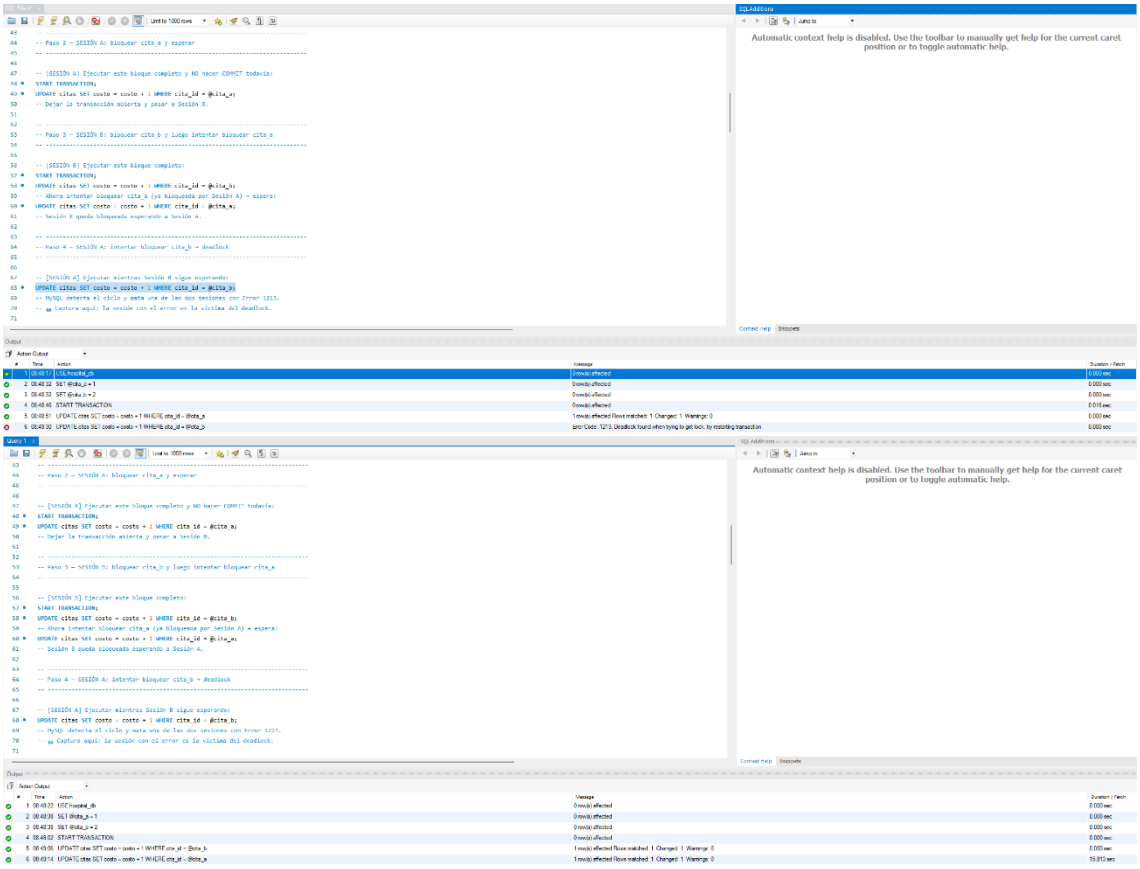
Etapa 5: Concurrencia y transacciones

Scripts usados: **07_vistas.sql** y **08_seguridad.sql**

Se utilizaron dos sesiones de MySQL Workbench actualizando las mismas filas de la tabla **citas** en orden inverso, generando un bloqueo circular.

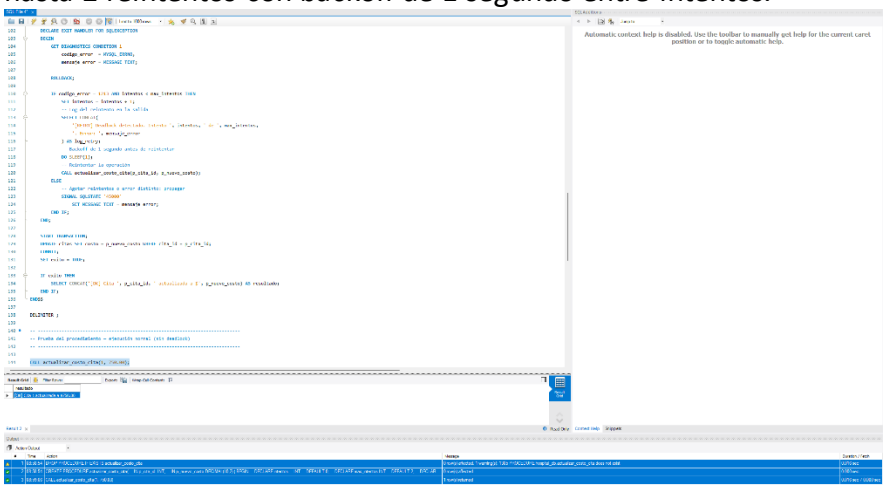
Paso	Sesión A	Sesión B
1	`START TRANSACTION`	
2	`UPDATE citas ... WHERE cita_id = 1` → bloquea fila 1	
3		`START TRANSACTION`
4		`UPDATE citas ... WHERE cita_id = 2` → bloquea fila 2
5		`UPDATE citas ... WHERE cita_id = 1` → **espera**
6	`UPDATE citas ... WHERE cita_id = 2` → **Error 1213**	

MySQL detectó el ciclo y eligió a **Sesión A** como víctima. **Sesión B** completó su UPDATE tras 15.8 segundos de espera.



Transacciones con retry ante deadlock

Se implementó el procedimiento **actualizar_costo_cita(p_cita_id INT, p_nuevo_costo DECIMAL)** con **START TRANSACTION / COMMIT / ROLLBACK** y manejo del Error 1213: hasta 2 reintentos con backoff de 1 segundo entre intentos.



Comparación de niveles de aislamiento

Escenario: Sesión A lee **costo** de **cita_id = 1**, Sesión B lo modifica y hace COMMIT, Sesión A vuelve a leer.

Nivel	Primera lectura	Commit de Sesión B	Segunda lectura	Comportamiento
READ COMMITTED	750.00	costo = 9999.00	9999.00	Lectura no repetible
REPEATABLE READ	750.00	costo = 9999.00	750.00	Snapshot del inicio

Con **READ COMMITTED**, Sesión A vio el valor modificado en la segunda lectura. Con **REPEATABLE READ** (nivel por defecto de InnoDB), Sesión A mantuvo el valor original durante toda la transacción.

Informe de observaciones

El deadlock se produjo de forma inmediata al generarse el bloqueo circular: MySQL no esperó el timeout sino que detectó el ciclo en milisegundos y eligió una víctima. El mecanismo de retry implementado en el procedimiento **actualizar_costo_cita** permite que la operación se reintente automáticamente ante este error, evitando que llegue a la capa de aplicación.

La diferencia entre **READ COMMITTED** y **REPEATABLE READ** fue claramente observable: con el primer nivel la segunda lectura devolvió el valor actualizado por otra sesión (9999.00), mientras que con el segundo nivel la transacción mantuvo el valor original (750.00) durante toda su duración. Para un sistema hospitalario, donde la consistencia dentro de una operación es crítica (por ejemplo, verificar disponibilidad de una sala antes de confirmar una cita), **REPEATABLE READ** es el nivel de aislamiento más adecuado, y es además el nivel por defecto de MySQL/InnoDB.

Nota: Mismo caso que las capturas de la etapa 1 y 3, *el Anexo IA se adjunta en un documento complementario: Anexo_IA_Emperador.pdf y el video*